

Proyecto de Fin de Grado: SmartFit

Desarrollo de Aplicación Multiplataforma



Juan Lerma Martinez

Febrero 2026

Abstract

SmartFit es una aplicación Android desarrollada en Kotlin con Jetpack Compose que permite a los usuarios llevar un seguimiento completo de su actividad física y nutricional. La aplicación ofrece gestión de rutinas de entrenamiento, registro de dietas y alimentos, seguimiento de medidas corporales y visualización del progreso mediante gráficas. El backend está desarrollado con FastAPI y Python, conectado a una base de datos MySQL desplegada en Amazon RDS, con la API alojada en una instancia EC2 de AWS. La arquitectura sigue el patrón MVVM y consume una API REST para la comunicación cliente-servidor, incorporando además notificaciones programadas mediante WorkManager para recordar al usuario su hora de entrenamiento.

Abstract

SmartFit is an Android application developed in Kotlin with Jetpack Compose that allows users to comprehensively track their physical activity and nutrition. The app provides workout routine management, diet and food logging, body measurement tracking, and progress visualization through charts. The backend is built with FastAPI and Python, connected to a MySQL database hosted on Amazon RDS, with the API deployed on an AWS EC2 instance. The architecture follows the MVVM pattern and consumes a REST API for client-server communication, also incorporating scheduled notifications via WorkManager to remind the user of their training time.

Contents

1	Introducción y Justificación	4
1.1	Descripción del Proyecto	4
1.2	Justificación del impacto social o ambiental, problema que resuelve. .	4
1.3	Identificación de los módulos implicados y su contribución.	5
1.4	Previsión de datos, pantallas y tecnologías a utilizar.	5
2	Análisis de Requisitos	7
2.1	Acceso a datos	7
2.2	Desarrollo de interfaces	8
2.3	Programación multimedia y móviles	9
2.4	Programación de servicios y procesos	9
2.5	Sistemas de gestión empresarial	10
2.6	Sostenibilidad aplicada al sistema productivo	10
3	Diseño del sistema	11
3.1	Diseño de la Base de Datos	11
3.2	Diccionario de Datos	12
3.2.1	Tabla: USUARIO	12
3.2.2	Tabla: COMIDA	13
3.2.3	Tabla: DIETA	13
3.2.4	Tabla: DIETA COMIDA (Tabla Intermedia)	14
3.2.5	Tabla: MEDIDA CORPORAL	14
3.2.6	Tabla: RUTINA	15
3.2.7	Tabla: EJERCICIO	15
3.2.8	Tabla: RUTINA EJERCICIO (Tabla Intermedia)	15
3.2.9	Tabla: HISTORIAL ENTRENAMIENTO	15
3.3	Diseño de la Interfaz de Usuario	16
3.3.1	Principios de Diseño	16
3.3.2	Bocetos y Navegación	16

4	Tecnologías y Herramientas	18
4.1	Lenguajes de Programación	18
4.2	Frameworks y Librerías	18
4.3	Base de Datos	19
4.4	Herramientas de Desarrollo y Control de Versiones	19
5	Implementación	20
5.1	Estructura del Proyecto	20
5.1.1	Backend	20
5.1.2	Frontend Android	20
5.2	Arquitectura MVVM	21
5.3	Descripción de los Módulos Principales	21
5.3.1	Módulo de Autenticación	21
5.3.2	Módulo de Rutinas y Ejercicios	21
5.3.3	Módulo de Dietas	21
5.3.4	Módulo de Progreso	22
5.3.5	Módulo de Notificaciones	22
5.3.6	Gráfico circular personalizado con Canvas	22
5.4	Funcionalidades Clave Implementadas	22
5.5	Capturas de Pantalla de la Aplicación	23
6	Pruebas	27
6.1	Plan de Pruebas	27
6.2	Pruebas Unitarias	27
6.3	Pruebas de Interfaz de Usuario	28
6.4	Errores Detectados y Soluciones	28
6.4.1	Errores de Compilación Kotlin	29
6.4.2	Errores de API (500)	29
6.4.3	Errores de Lógica	29
6.4.4	Errores de Base de Datos	30
6.4.5	Errores de UI/UX	30
6.4.6	Errores de Navegación	30
6.4.7	Errores de Arquitectura y ViewModel	31
6.4.8	Errores de Endpoints	31
7	Despliegue	32
7.1	Arquitectura de Despliegue en AWS	32
7.2	Requisitos de Instalación	32
7.3	Proceso de Despliegue del Backend	33
7.4	Proceso de Despliegue en Móvil	34

7.5	Manual Básico de Usuario	34
8	Planificación y Gestión del Proyecto	35
8.1	Metodología de Desarrollo	35
8.2	Cronograma del Proyecto	35
8.3	Competencias personales desarrolladas	35
8.3.1	Resolución de problemas	36
8.3.2	Comunicación técnica	36
8.3.3	Trabajo autónomo y cumplimiento de plazos	36
9	Conclusiones	37
9.1	Objetivos Alcanzados	37
9.2	Dificultades Encontradas	37
9.3	Mejoras Futuras	38
10	Anexos	39
10.1	Manual Técnico	39
10.1.1	Requisitos del Sistema	39
10.1.2	Variables de Entorno	39
10.2	Enlace al Repositorio	39
10.3	Referencias y Recursos Utilizados	39
11	Bibliografía	41
11.1	Referencias Bibliográficas en Formato APA	41

Chapter 1

Introducción y Justificación

1.1 Descripción del Proyecto

Mi proyecto consistiría en una aplicación para poder llevar un seguimiento de los avances en nuestro físico. La aplicación contará con un calendario para programar y visualizar los entrenamientos y las comidas registradas junto con rutinas predefinidas para principiantes todo esto, también dispondría de un apartado para apuntar los pesos utilizados en entrenamientos anteriores y las medidas del cuerpo en caso de querer conocerlas.

1.2 Justificación del impacto social o ambiental, problema que resuelve.

Mi proyecto consiste en una aplicación que busca solucionar un problema común en el ámbito del bienestar personal: la falta de una herramienta centralizada y accesible para el seguimiento de la salud y el fitness. La aplicación facilitará que las personas inicien y mantengan un cambio físico, mejorando así su salud general.

Actualmente, los usuarios se ven obligados a utilizar múltiples aplicaciones, para registrar sus entrenamientos, contar calorías y registrar medidas corporales. Esta fragmentación de la información dificulta tener una visión clara del progreso. Además, la aplicación aborda la barrera que enfrentan los principiantes al carecer de conocimiento sobre cómo empezar, resolviendo este problema al ofrecer rutinas predefinidas. Por último, al permitir a los usuarios registrar y visualizar su progreso de manera sencilla a través de un calendario, la aplicación combate la falta de motivación y consistencia. De esta manera, la aplicación tendrá un impacto directo al fomentar un estilo de vida más saludable. Esto se alinea con el Objetivo de Desarrollo Sostenible (ODS)

número 3: Salud y Bienestar, ya que contribuye a mejorar la salud física de los individuos y promueve hábitos saludables. Usuarios a los que está dirigido

Esta aplicación esta diseñada para cualquier usuario que quiera mejorar su fisico o su alimentacion, independientemente de si es un principiante o un usuario con mas experiencia.

Para los principiantes, la aplicación resuelve el problema de "no saber por dónde empezar", ofreciéndoles un punto de partida claro con rutinas predefinidas y un sistema de seguimiento intuitivo que fomenta la consistencia desde el primer día. Para los usuarios con más experiencia, la aplicación sirve como un cuaderno de bitácora digital. Les permite registrar con precisión sus entrenamientos, contar calorías para optimizar su nutrición y apuntar medidas corporales para monitorizar su progreso de forma detallada, lo cual es fundamental para alcanzar metas más específicas como ganar masa muscular o perder grasa.

1.3 Identificación de los módulos implicados y su contribución.

La aplicación tiene una implementación de una base de datos relacional con al menos cuatro tablas interrelacionadas Usuarios, Rutinas, Alimentos y Medidas Corporales. Para el Desarrollo de Interfaces, se diseñará una GUI clara y sencilla con menús coherentes e intuitivos. Se generarán informes con gráficos para que el usuario pueda ver su progreso de forma visual. Para el módulo de Programación Multimedia y Móviles, la aplicación tendrá un diseño adaptable que permitirá su uso tanto en teléfonos móviles como en ordenadores. La Programación de Servicios y Procesos se encargará de la comunicación cliente-servidor. También se aplicarán criterios de seguridad básicos como el cifrado de contraseñas. El proyecto también cubrirá el módulo de Sistemas de Gestión Empresarial a través del control de usuarios, la generación de informes de actividad y un panel de control básico que ofrezca un resumen del progreso personal. Finalmente, la aplicación está alineada con el ODS 3: Salud y Bienestar. A través de su uso, se fomenta un estilo de vida saludable.

1.4 Previsión de datos, pantallas y tecnologías a utilizar.

El proyecto gestionará la información del usuario a través de una base de datos. Los principales datos que se almacenarán incluyen la información personal del usuario,

los detalles de sus rutinas de ejercicio, un registro de su alimentación y sus medidas corporales, así como un historial de su actividad física. La aplicación se estructurará en varias pantallas, diseñadas para una navegación intuitiva y accesible en cualquier dispositivo. Se contará con un Panel de Control que mostrará un resumen del progreso del usuario. Habrá pantallas dedicadas para la gestión de Rutinas, el Registro de Alimentos, el seguimiento de Medidas y la visualización de Informes y Gráficos. Para el desarrollo del proyecto, se proponen tecnologías que se adaptan a un entorno multiplataforma. Para el backend se utilizará un lenguaje de programación de los utilizados en clase. El frontend se desarrollará con un framework que permita la compatibilidad con PC y móvil a través de un diseño adaptable.

Chapter 2

Análisis de Requisitos

2.1 Acceso a datos

- **Uso de una base de datos relacional con al menos 3–4 tablas relacionadas.**
 - **Usuarios:** Almacenar los datos personales de las personas que se registren.
 - **Rutinas:** Guardaran los ejercicios agrupados por rutinas que podran ser creadas por los usuarios.
 - **Ejercicios:** Almacenara todos los ejercicios para ser incluidos en las rutinas.
 - **Medidas Corporales:** Guardar la información de las medidas corporales de los usuarios.
 - **Comidas:** Almacenara las comidas de los usuarios.
 - **Dietas:** Almacenara las dietas de los usuarios.
 - **Historial de entrenamiento:** Guardar los datos de los entrenamientos.
- **Implementación de operaciones CRUD (crear, leer, actualizar y borrar).**
 - El proyecto implica la gestión de datos del usuario, incluyendo detalles de sus rutinas de ejercicio, el registro de su alimentación, sus medidas corporales, y un historial de actividad física, lo cual exige la capacidad de Crear, Leer, Actualizar y Borrar esta información en la base de datos.
- **Posibilidad de importar o exportar datos en formato JSON o XML.**
 - Los informes se generarán a partir de los datos almacenados en la base de datos, con el objetivo de ofrecer al usuario una visión de su progreso.

Los principales informes serán:

- * **Informe de evolución física:** Mostrará gráficas con la variación del peso y medidas a lo largo del tiempo.
- * **Informe de actividad y entrenamientos:** Incluirá estadísticas sobre la frecuencia de los entrenamientos, el número de rutinas completadas y el tipo de ejercicios más realizados.
- * **Informe nutricional:** Reflejará las calorías consumidas, así como la proporción de macronutrientes en la dieta.

2.2 Desarrollo de interfaces

- **Una interfaz gráfica de usuario (GUI) clara, usable y accesible.**
 - La aplicación contará con una interfaz gráfica, intuitiva y centrada en la experiencia del usuario, diseñada para facilitar el seguimiento diario del entrenamiento, la alimentación y las medidas corporales.
 - La pantalla principal se presentará como un calendario, donde el usuario podrá visualizar los días de entrenamiento realizados y los planificados, así como acceder rápidamente a la información de cada jornada.
 - Además, la interfaz se organizará en diferentes secciones especializadas:
 - * **Medidas corporales:** registro y seguimiento con gráficos de evolución física.
 - * **Comidas:** registro de alimentos, cálculo de calorías y equilibrio nutricional.
 - * **Gestión de rutinas:** edición y modificación centralizada de rutinas y ejercicios.
 - El diseño tendrá una distribución clara, iconografía intuitiva y colores contrastados que garanticen la legibilidad para ofrecer una experiencia fluida.
- **Inclusión de formularios y menús coherentes, con validación básica.**
 - Se dispondrá de formularios simples para introducir datos en cada módulo (comidas, rutinas y medidas), incorporando validaciones como campos obligatorios y formatos correctos.

- Los menús seguirán una estructura coherente, con botones situados en lugares familiares para el usuario, favoreciendo la rapidez de uso y reduciendo la curva de aprendizaje.
- **Generación de al menos un informe con gráficos a partir de los datos almacenados.**
 - La aplicación mostrará visualmente el progreso mediante diversos tipos de gráficas:
 - * **Gráfico de barras:** para comparar pesos levantados y repeticiones a lo largo del tiempo.
 - * **Gráfico de línea:** para observar la evolución del rendimiento en ejercicios específicos semana a semana.
 - * **Gráficos combinados:** para facilitar la comparación y análisis del rendimiento general.

2.3 Programación multimedia y móviles

- **Ejecución en ordenador y versión móvil (diseño adaptable).**
 - **Versión para PC:** permitirá un uso completo, incluyendo el ingreso de datos y la visualización de informes en pantalla grande.
 - **Versión móvil:** el diseño será *responsive*, ajustándose a móviles y tabletas para facilitar el registro de datos en el propio gimnasio.
- **Uso de funcionalidades multimedia.**
 - El proyecto incluye la visualización de informes, gráficos y un panel de control visual para resumir el progreso.
- **Soporte de conexión HTTPS y seguridad.**
 - Se garantiza la protección mediante cifrado de contraseñas y protocolos seguros como HTTPS para la comunicación cliente-servidor, evitando la interceptación de información sensible.

2.4 Programación de servicios y procesos

- **Mecanismo de comunicación cliente-servidor (API REST).**

- Se incluirá una API sencilla para que los entrenadores personales puedan introducir rutinas personalizadas para sus clientes.
- **Gestión de procesos o hilos para tareas en segundo plano.**
 - El sistema utilizará procesos en segundo plano para enviar recordatorios y notificaciones automáticas (hora de entrenar, registro de comidas), mejorando la constancia del usuario.
- **Criterios básicos de seguridad.**
 - Implementación de un sistema de roles (usuario normal vs. entrenador) y almacenamiento de contraseñas cifradas.

2.5 Sistemas de gestión empresarial

- **Control de usuarios y clientes.**
 - Gestión diferenciada de clientes y entrenadores, donde los primeros consultan su progreso y los segundos administran las rutinas de sus clientes.
- **Generación de informes de actividad y panel de control.**
 - Los informes incluirán pesos, repeticiones, duración y frecuencia de ejercicio.
 - Se incluirá un *dashboard* o panel de control básico con el resumen del progreso personal.

2.6 Sostenibilidad aplicada al sistema productivo

- **Vínculo con los Objetivos de Desarrollo Sostenible (ODS).**
 - Alineación con el ODS número 3: Salud y Bienestar, promoviendo hábitos saludables y mejora física.
- **Reducción del impacto ambiental.**
 - Al funcionar como un cuaderno digital, se elimina el uso de papel.
 - Eficiencia mediante código optimizado para reducir el consumo de recursos y facilitar el mantenimiento futuro mediante la reutilización de componentes.

Chapter 3

Diseño del sistema

3.1 Diseño de la Base de Datos

Para la persistencia de la información de SmartFit, se ha diseñado un modelo entidad-relación que garantiza la integridad de los datos y evita redundancias.

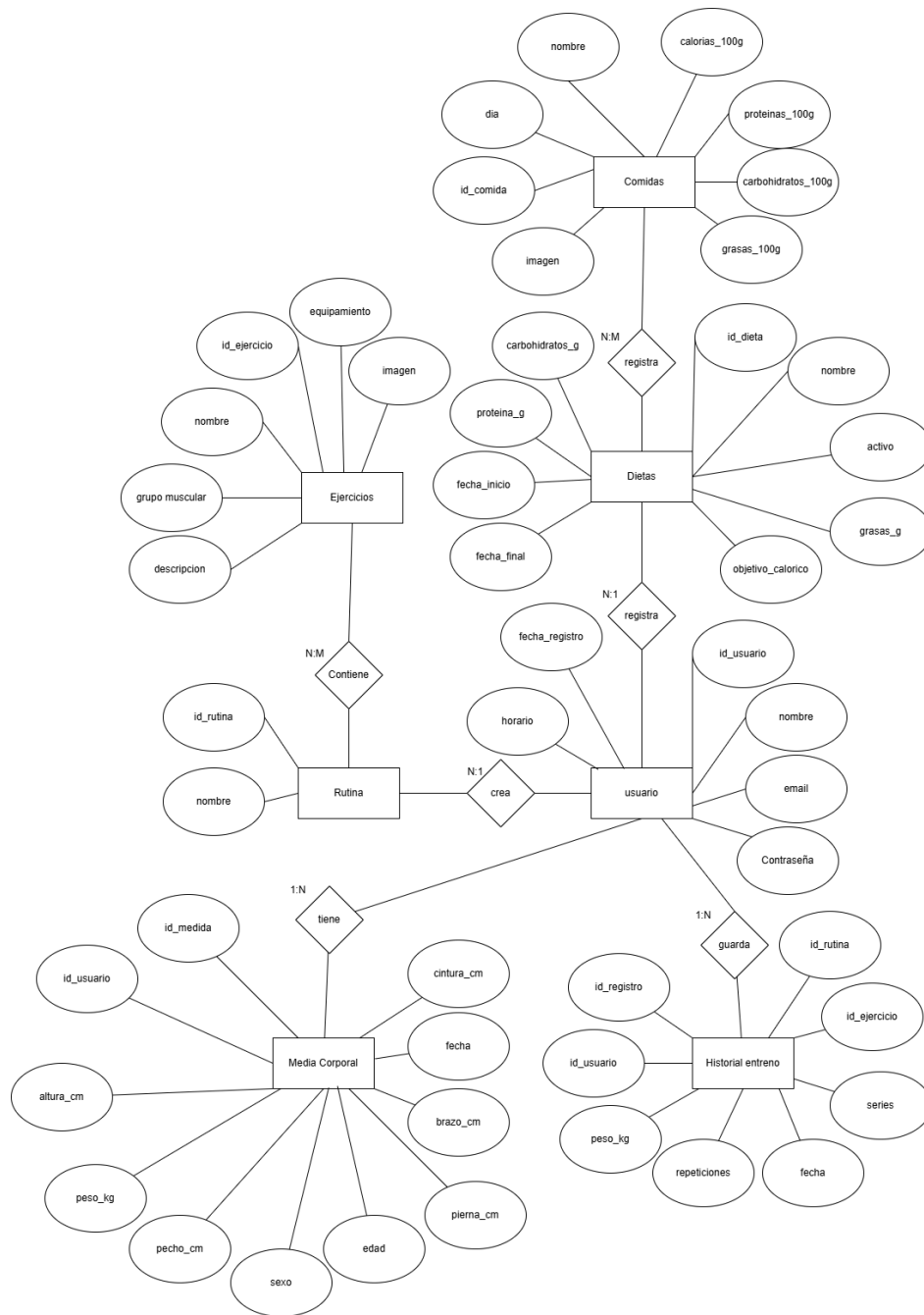


Figure 3.1: Modelo Entidad-Relación de la base de datos SmartFit.

3.2 Diccionario de Datos

En esta sección se describe la estructura técnica de la base de datos de SmartFit, detallando cada campo, su tipo de dato y su función.

3.2.1 Tabla: USUARIO

Almacena los perfiles y datos de acceso de los usuarios.

Campo	Tipo	Nulo	Descripción
id_usuario	INT (PK)	No	Identificador único autoincremental.
nombre	VARCHAR(100)	No	Nombre completo del usuario.
email	VARCHAR(100)	No	Correo electrónico único (login).
password_hash	VARCHAR(255)	No	Contraseña cifrada por seguridad.
horario_entrenamiento	TIME	Sí	Hora a la que se quiere ir a entrenar
fecha_registro	DATETIME	No	Fecha y hora automática del alta.

3.2.2 Tabla: COMIDA

Catálogo de alimentos y sus valores nutricionales de referencia por cada 100g.

Campo	Tipo	Nulo	Descripción
id_comida	INT (PK)	No	Identificador único del alimento.
nombre	VARCHAR(100)	No	Nombre del alimento.
calorias_100g	INT	No	Energía por cada 100 gramos.
proteinas_100g	INT	No	Proteínas por cada 100 gramos.
carbohidratos_100g	INT	No	Carbohidratos por cada 100 gramos.
grasas_100g	INT	No	Grasas por cada 100 gramos.
id_usuario	INT (FK)	Sí	Creador del alimento (si es personalizado).
id_dieta	INT (FK)	Sí	Dieta a la que se asocia el alimento.
imagen	VARCHAR(255)	Sí	Ruta de la imagen del alimento.

3.2.3 Tabla: DIETA

Define los objetivos nutricionales programados para un usuario.

Campo	Tipo	Nulo	Descripción
id_dieta	INT (PK)	No	Identificador único de la dieta.
nombre	VARCHAR(100)	No	Nombre del plan nutricional.
objetivo_calorico	INT	No	Meta calórica diaria.
proteinas_g	DECIMAL(5,2)	Sí	Meta de proteínas en gramos.
carbohidratos_g	DECIMAL(5,2)	Sí	Meta de carbohidratos en gramos.
grasas_g	DECIMAL(5,2)	Sí	Meta de grasas en gramos.
fecha_inicio	DATE	No	Fecha de inicio del plan.
fecha_fin	DATE	Sí	Fecha de finalización prevista.
id_usuario	INT (FK)	No	Usuario propietario de la dieta.

3.2.4 Tabla: DIETA COMIDA (Tabla Intermedia)

Gestiona qué comidas pertenecen a cada dieta

Campo	Tipo	Nulo	Descripción
id	INT (PK)	No	ID único de la relación dieta-comida.
id_dieta	INT (FK)	No	Referencia a la dieta asignada.
id_comida	INT (FK)	No	Referencia a la comida vinculada.
tipo	VARCHAR(50)	No	Momento del día (ej. Desayuno, Almuerzo).
dia	VARCHAR(20)	No	Día de la semana asignado a la ingesta.

3.2.5 Tabla: MEDIDA CORPORAL

Historial de seguimiento biométrico detallado del usuario.

Campo	Tipo	Nulo	Descripción
id_medida	INT (PK)	No	Identificador del registro físico.
id_usuario	INT (FK)	No	Usuario asociado.
fecha	DATE	No	Fecha de la toma de medidas.
peso_kg	DECIMAL(5,2)	No	Peso corporal en kilogramos.
altura_cm	DECIMAL(5,2)	Sí	Altura del usuario en centímetros.
pecho_cm	DECIMAL(5,2)	Sí	Perímetro del pecho.
pierna_cm	DECIMAL(5,2)	Sí	Perímetro de la pierna.
brazo_cm	DECIMAL(5,2)	Sí	Perímetro del brazo.
grasa_corporal_pct	DECIMAL(4,2)	Sí	Porcentaje de grasa estimado.

3.2.6 Tabla: RUTINA

Define las sesiones de entrenamiento creadas por los usuarios.

Campo	Tipo	Nulo	Descripción
id_rutina	INT (PK)	No	Identificador único de la rutina.
nombre	VARCHAR(100)	No	Nombre descriptivo de la sesión.
id_usuario	INT (FK)	No	Usuario propietario de esta rutina específica.

3.2.7 Tabla: EJERCICIO

Catálogo maestro de todos los ejercicios disponibles en la aplicación.

Campo	Tipo	Nulo	Descripción
id_ejercicio	INT (PK)	No	Identificador único del ejercicio.
nombre	VARCHAR(255)	No	Nombre técnico del ejercicio.
grupo_muscular	VARCHAR(100)	Sí	Músculo principal trabajado.
equipamiento	VARCHAR(100)	Sí	Material necesario (Mancuerna, barra, etc.).
descripcion	VARCHAR(250)	Sí	Breve guía sobre la técnica de ejecución.
imagen	VARCHAR(255)	Sí	Nombre o ruta del archivo de imagen ilustrativa.

3.2.8 Tabla: RUTINA EJERCICIO (Tabla Intermedia)

Gestiona qué ejercicios pertenecen a cada rutina y sus parámetros.

Campo	Tipo	Nulo	Descripción
id_rutina	INT (PFK)	No	Relación con la tabla RUTINA.
id_ejercicio	INT (PFK)	No	Relación con la tabla EJERCICIO.
series	INT	No	Cantidad de series (Def: 3).
repeticiones	INT	No	Cantidad de repeticiones (Def: 10).
orden	INT	Sí	Orden de ejecución en la sesión.

3.2.9 Tabla: HISTORIAL ENTRENAMIENTO

Registro de la actividad física real ejecutada.

Campo	Tipo	Nulo	Descripción
id_registro	INT (PK)	No	ID único del registro de serie.
id_usuario	INT (FK)	No	Usuario que ejecutó la serie.
id_ejercicio	INT (FK)	No	Ejercicio realizado.
id_rutina	INT	No	Rutina realizada.
peso_kg	FLOAT	No	Peso real levantado.
repeticiones	INT	No	Repeticiones completadas.
series	INT	No	Número de serie dentro de la sesión.
duracion_minutos	INT	No	Duración del entrenamiento
fecha	DATETIME	No	Momento del registro (Timestamp).

3.3 Diseño de la Interfaz de Usuario

El diseño de la interfaz de SmartFit se ha basado en los principios de **Material Design 3**, buscando una experiencia de usuario (UX) limpia, moderna y, sobre todo, funcional para un entorno de entrenamiento.

3.3.1 Principios de Diseño

- **Accesibilidad:** Uso de contrastes elevados y tipografías legibles para facilitar la lectura en condiciones de fatiga física.
- **Navegación Intuitiva:** Implementación de una barra de navegación inferior (*Bottom Navigation Bar*) para acceder rápidamente a las secciones de Calendario, Rutinas, Comidas y Perfil.
- **Feedback Visual:** Uso de componentes interactivos que confirman las acciones del usuario (ej. *Snackbars* tras guardar una serie al fallo).

3.3.2 Bocetos y Navegación

A continuación, se presentan los esquemas visuales que definen el flujo de la aplicación.

Wireframes

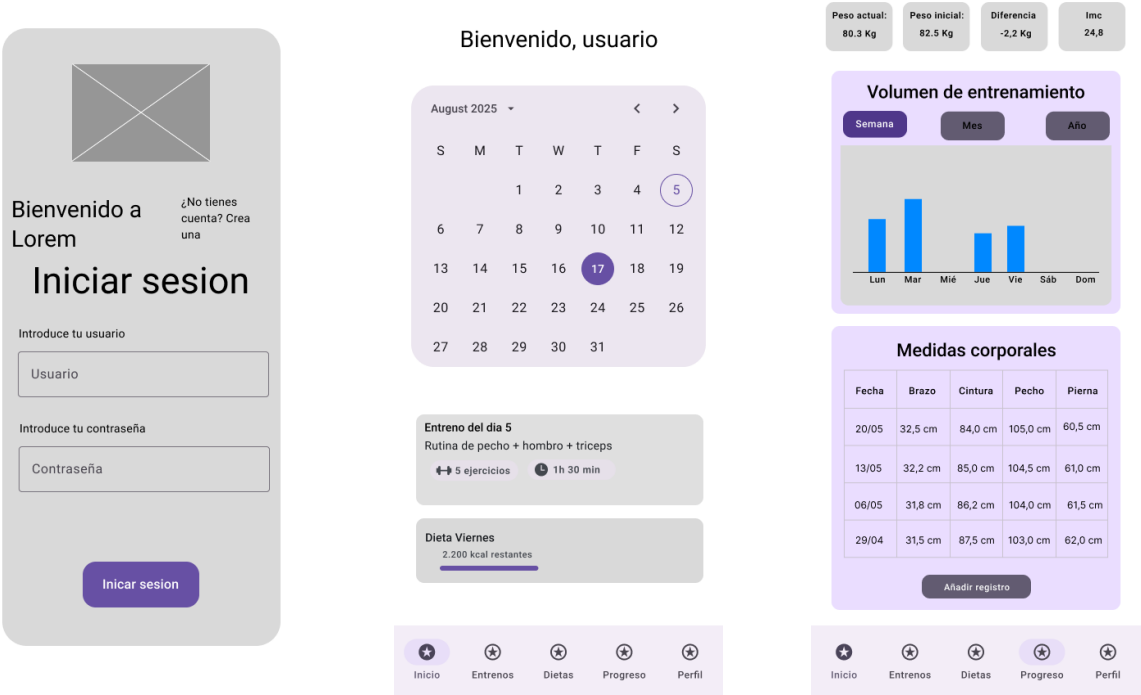


Figure 3.2: Wireframes de las pantallas de Login, Inicio y Progreso.

Prototipo de Alta Fidelidad

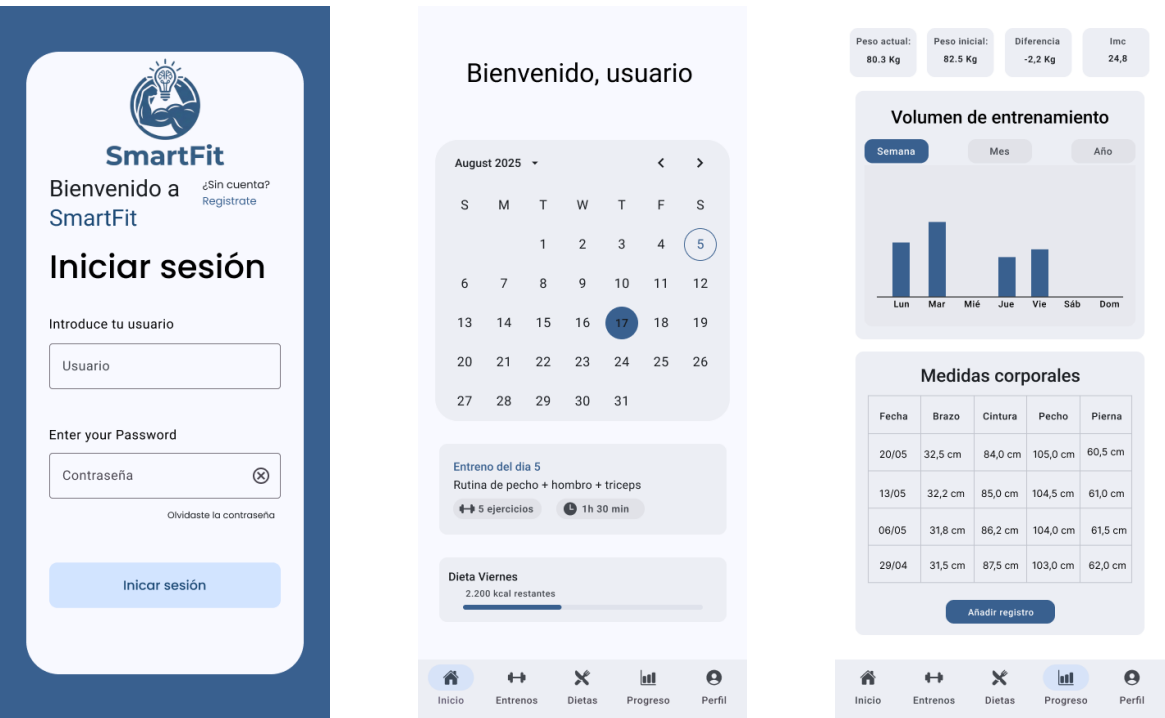


Figure 3.3: Prototipo final en Figma: Login, Inicio y Progreso.

Chapter 4

Tecnologías y Herramientas

En este capítulo se detallan los recursos tecnológicos seleccionados para el desarrollo de SmartFit, justificando su elección en base a los requisitos de rendimiento y multiplataforma del proyecto.

4.1 Lenguajes de Programación

- **Python:** Utilizado como lenguaje principal para el desarrollo del *backend*. Su sintaxis clara y su potente ecosistema de librerías permiten un desarrollo ágil y un mantenimiento sencillo.
- **Kotlin:** Empleado para el desarrollo de la vertiente móvil de la aplicación. Se ha seleccionado por su total interoperabilidad con el ecosistema Android, su seguridad frente a errores de punteros nulos (*null safety*) y su expresividad, lo que facilita la creación de interfaces robustas y fluidas.
- **SQL:** Lenguaje de consulta estructurado empleado para la definición, manipulación y gestión de la base de datos relacional en el servidor MySQL.

4.2 Frameworks y Librerías

- **FastAPI:** Framework web moderno y de alto rendimiento para construir APIs con Python basado en anotaciones de tipo estándar. Se ha elegido por su velocidad y su generación automática de documentación interactiva (Swagger).
- **SQLAlchemy:** *Object-Relational Mapper* (ORM) que permite interactuar con la base de datos mediante objetos Python, abstrayendo la complejidad de las consultas manuales y mejorando la seguridad contra inyecciones SQL.

- **Pydantic:** Utilizado para la validación de datos y la gestión de esquemas, garantizando que la información que entra y sale de la API sea correcta.
- **Python-jose & Passlib:** Librerías fundamentales para la seguridad, encargadas de la gestión de *JSON Web Tokens* (JWT) y el cifrado de contraseñas mediante algoritmos de hash seguros (bcrypt).

4.3 Base de Datos

Para la persistencia de datos se ha optado por **MySQL**, un sistema de gestión de bases de datos relacionales robusto y escalable. Su integración con SQLAlchemy permite una gestión eficiente de las relaciones $N : M$ implementadas en el sistema.

4.4 Herramientas de Desarrollo y Control de Versiones

- **Visual Studio Code:** Entorno de desarrollo integrado (IDE) principal, ampliado con extensiones para Python y bases de datos.
 - *Claude Code:* Herramienta de asistencia por IA integrada en la terminal para la refactorización de código y resolución de errores en tiempo real.
- **Git & GitHub:** Sistema de control de versiones distribuido para el seguimiento del código y el almacenamiento del repositorio remoto.
- **L^AT_EX:** Empleado para la redacción de la presente memoria técnica, garantizando un acabado profesional y una gestión eficiente de bibliografías y referencias.

Chapter 5

Implementación

5.1 Estructura del Proyecto

El proyecto está dividido en dos partes diferenciadas: el **backend** (API REST) y el **frontend** (aplicación Android).

5.1.1 Backend

El backend se organiza en los siguientes archivos principales:

- **main.py**: Punto de entrada de la aplicación FastAPI. Configura la conexión a la base de datos y registra los routers.
- **models.py**: Define las entidades de la base de datos mediante SQLAlchemy ORM.
- **schemas.py**: Define los esquemas Pydantic para la validación de datos de entrada y salida de la API.
- **routers.py**: Contiene todos los endpoints REST organizados por recurso.
- **.env**: Almacena las variables de entorno sensibles (credenciales de BD, clave secreta JWT).

5.1.2 Frontend Android

La aplicación Android sigue una estructura de paquetes basada en la arquitectura MVVM:

- **ui/screens/**: Pantallas Composable organizadas por módulo funcional.
- **ui/viewmodel/**: ViewModels que gestionan el estado de cada pantalla.

- **network/**: Capa de red con Retrofit, modelos de respuesta y SessionManager.
- **navigation/**: NavGraph y definición de rutas de navegación.
- **notificaciones/**: Worker, Scheduler y Helper para las notificaciones push.

5.2 Arquitectura MVVM

La aplicación Android implementa el patrón **Model-View-ViewModel (MVVM)**, separando claramente las responsabilidades:

- **Model**: Los datos provienen de la API REST a través de Retrofit. Los modelos de red se definen en `ApiModels.kt`.
- **View**: Las pantallas están implementadas con Jetpack Compose. Observan el estado del ViewModel mediante `StateFlow` y `collectAsState()`.
- **ViewModel**: Gestiona la lógica de negocio, realiza las llamadas a la API en corrutinas y expone el estado a la vista mediante `MutableStateFlow`.

5.3 Descripción de los Módulos Principales

5.3.1 Módulo de Autenticación

La autenticación se gestiona mediante JWT. Al iniciar sesión, el servidor devuelve un token que se almacena en `SharedPreferences` a través del `SessionManager`. Las contraseñas se almacenan cifradas con `bcrypt` en el servidor.

5.3.2 Módulo de Rutinas y Ejercicios

Permite crear, editar y eliminar rutinas de entrenamiento. Cada rutina puede contener múltiples ejercicios con sus series y repeticiones. Al finalizar un entrenamiento, los datos se guardan en el historial mediante una llamada POST al endpoint `/historial`.

5.3.3 Módulo de Dietas

Permite crear planes dietéticos semanales asignando alimentos por día y tipo de comida (desayuno, comida, cena). Los macronutrientes se calculan automáticamente en el `ViewModel` a partir de los alimentos añadidos.

5.3.4 Módulo de Progreso

Muestra la evolución del usuario mediante gráficas de barras del volumen de entrenamiento (peso × series × repeticiones) y una tabla con el historial de medidas corporales.

5.3.5 Módulo de Notificaciones

Implementado con **WorkManager**, que ejecuta el trabajo en un hilo secundario (`CoroutineWorker`). El `NotificationScheduler` calcula el retardo hasta 30 minutos antes de la hora de entrenamiento configurada y programa el `EntrenamientoWorker`, que lanza la notificación al usuario.

5.3.6 Gráfico circular personalizado con Canvas

Para mostrar las calorías restantes en la pantalla de dieta activa, se optó por un componente personalizado dibujado con `Canvas` en lugar del `CircularProgressIndicator` estándar de Material 3. Esta decisión permite un control total sobre el aspecto visual: grosor del trazo, color, punto de inicio y comportamiento cuando el progreso es cero.

El componente dibuja dos arcos superpuestos:

- **Arco de fondo (track):** siempre completo (360°), en color `surfaceContainerHigh`, actúa como pista visual.
- **Arco de progreso:** proporcional al valor `progreso` (entre 0.0 y 1.0), en color primario, solo se dibuja si hay progreso real para evitar un arco de longitud cero.

Ambos arcos parten de `startAngle = -90f` (parte superior del círculo) y usan `StrokeCap.Round` para obtener extremos redondeados. Las calorías restantes se muestran centradas dentro del círculo mediante un `Box` con `contentAlignment = Alignment.Center`.

5.4 Funcionalidades Clave Implementadas

- Registro e inicio de sesión con cifrado `bcrypt` y tokens `JWT`.
- Configuración inicial al crear cuenta: peso, altura y hora de entrenamiento.
- CRUD completo de rutinas con asignación de ejercicios.
- Registro de entrenamientos con seguimiento de series y pesos.

- Creación de dietas semanales con alimentos por día y tipo de comida.
- Registro de medidas corporales con visualización de progreso.
- Notificaciones locales programadas mediante WorkManager.
- Soporte de modo oscuro y claro mediante Material Design 3.
- Persistencia de estado ante rotación de pantalla con `rememberSaveable`.
- Animaciones de expansión con física de muelle (*Spring*) en Material3.
- Despliegue de la API en AWS EC2 con base de datos en Amazon RDS.

5.5 Capturas de Pantalla de la Aplicación



Figure 5.1: Pantalla de inicio de sesión.

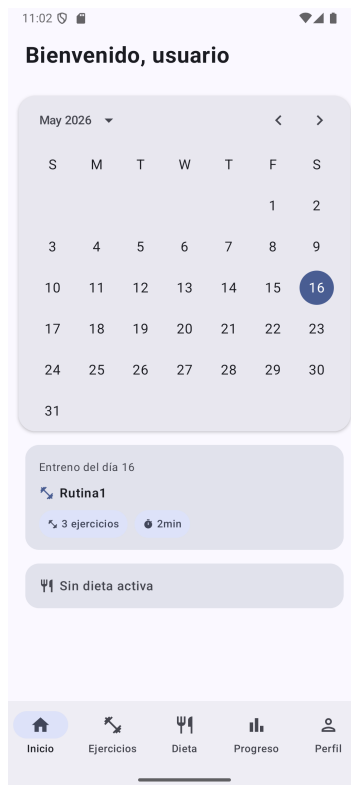


Figure 5.2: Pantalla principal (Home).

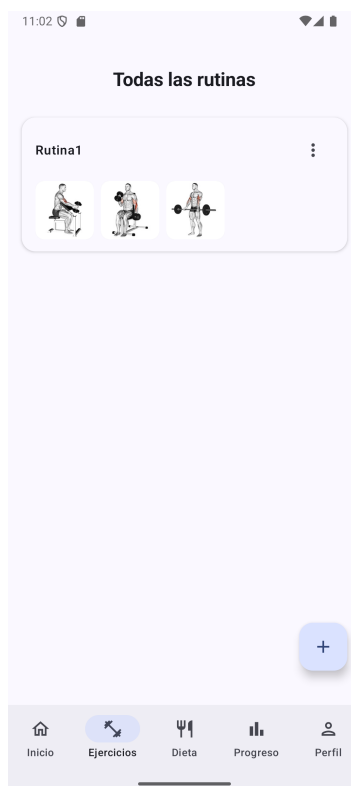


Figure 5.3: Pantalla de gestión de rutinas.

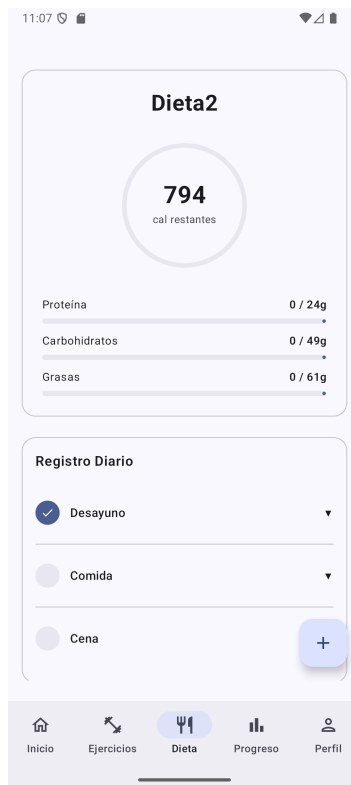


Figure 5.4: Pantalla de dieta activa.



Figure 5.5: Pantalla de progreso con gráfica de volumen.

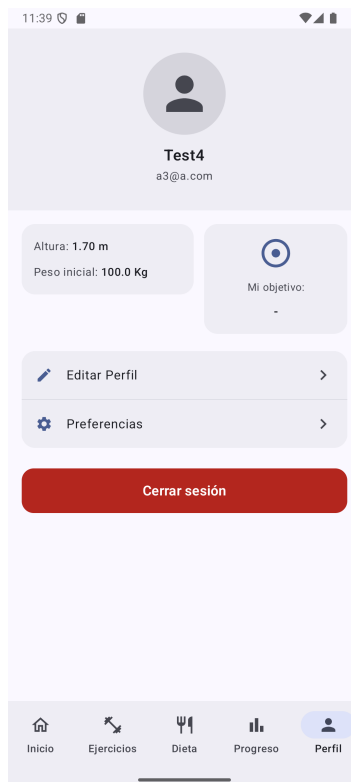


Figure 5.6: Pantalla de perfil del usuario.

Chapter 6

Pruebas

6.1 Plan de Pruebas

Se han realizado dos tipos de pruebas automatizadas siguiendo los criterios del módulo de Desarrollo de Interfaces: pruebas unitarias (*Unit Testing*) y pruebas de interfaz de usuario (*UI Testing*).

6.2 Pruebas Unitarias

Las pruebas unitarias validan la lógica de negocio de la clase `ListaComidasUiState`, concretamente la propiedad computada `comidasFiltradas`, que implementa el filtrado y ordenación de alimentos. Estas pruebas se ejecutan en la JVM sin necesidad de dispositivo o emulador.

Test	Descripción	Resultado
busquedaPorNombreFiltraCorrectamente	Verifica que la búsqueda por texto devuelve solo los alimentos cuyo nombre coincide.	PASS
busquedaInsensibleAMayusculas	Verifica que la búsqueda ignora mayúsculas y minúsculas.	PASS
filtroProteinasOrdenaDeMayorAMenor	Verifica que el filtro de proteínas ordena los alimentos de mayor a menor contenido proteico.	PASS
sinFiltroActivoNoOrdena	Verifica que sin filtro activo se devuelven los alimentos en el orden original.	PASS

Table 6.1: Resultados de las pruebas unitarias.

6.3 Pruebas de Interfaz de Usuario

Las pruebas de UI verifican el comportamiento de los componentes de la pantalla de inicio de sesión. Se ejecutan sobre un emulador Android mediante Compose Testing.

Test	Descripción	Resultado
loginScreen_camposVacios_muestraError	Verifica que al pulsar el botón de login con campos vacíos se muestran al menos dos campos de texto editables.	PASS
loginScreen_botonRegistro_esClickable	Verifica que el botón "Regístrate" ejecuta la acción de navegación correctamente.	PASS

Table 6.2: Resultados de las pruebas de interfaz de usuario.

6.4 Errores Detectados y Soluciones

Durante el desarrollo se detectaron los siguientes errores significativos:

6.4.1 Errores de Compilación Kotlin

- **Redeclaración de clases:** `AñadirEjercicioUiState` y `FiltroTipo` estaban declaradas en dos archivos simultáneamente. Solución: eliminar el archivo duplicado `ListaEjerciciosViewModel.kt`.
- **Imports incorrectos:** Algunos archivos usaban el paquete `com.tuapp` en lugar de `com.example.proyectazo`, y `SessionManager` se importaba en archivos que no debían usarlo. Solución: corregir el paquete y eliminar dependencias innecesarias.
- **Tipos de función incompatibles:** La firma `onAnadirEjercicio: () -> Unit` no aceptaba el parámetro `rutinaId` que se le pasaba. Solución: cambiar la firma a `onAnadirEjercicio: (rutinaId: Int) -> Unit`.
- **remember fuera de contexto Composable:** La función `clickableNoRipple` usaba `remember` sin ser una función `@Composable`. Solución: añadir la anotación `@Composable`.

6.4.2 Errores de API (500)

- **Campo en schema pero no en modelo:** `RutinaCreate` tenía el campo `descripcion` pero el modelo `SQLAlchemy` no lo incluía. Solución: eliminar `descripcion` de `schemas.py`.
- **id_rutina = 0 en historial:** El historial se guardaba con una clave foránea inválida. Solución: añadir la columna `id_rutina` a la tabla y al endpoint de retorno.
- **id_rutina no devuelta por la API:** El endpoint `historial_usuario` construía el diccionario manualmente sin incluir `id_rutina`. Solución: añadir el campo al return del endpoint.

6.4.3 Errores de Lógica

- **Rutina duplicada al añadir ejercicios:** Cada vez que se pulsaba “Añadir ejercicio” se creaba una rutina nueva. Solución: implementar el método `crearONavegar()` que guarda el `rutinaId` en memoria y lo reutiliza si ya existe.
- **userId = 0 pasado a la API:** El `userId` se leía antes del login, devolviendo siempre 0. Solución: leer el `userId` con `SessionManager` en el momento de entrar a la pantalla.

- **Timer de entrenamiento incorrecto:** `sumOf { it.duracion_minutos }` multiplicaba la duración por el número de ejercicios. Solución: usar `maxOf` ya que todos comparten la misma duración.
- **Ruta incorrecta en la barra de navegación:** El ítem “Progreso” apuntaba a `Screen.Dieta.route`. Solución: cambiar a `Screen.Progreso.route`.

6.4.4 Errores de Base de Datos

- **Safe update mode:** MySQL bloqueaba `UPDATES` sin cláusula `WHERE` con clave primaria. Solución: ejecutar `SET SQL_SAFE_UPDATES = 0` antes de la consulta.
- **FK constraint en UPDATE:** Se intentaba asignar un `id_rutina` que no existía en la tabla `RUTINA`. Solución: consultar primero un id válido antes del `UPDATE`.
- **Imágenes sin URL completa:** La BD almacenaba solo el nombre del archivo sin la URL de S3. Solución: actualizar con `CONCAT('https://...s3.../', imagen)` en MySQL.

6.4.5 Errores de UI/UX

- `mutableListOf` **no reactivo:** Añadir series no actualizaba la interfaz. Solución: cambiar a `mutableStateListOf` para que `Compose` detecte los cambios.
- **Subcarpetas en drawable:** Android no acepta subcarpetas dentro de `drawable/`. Solución: mover las imágenes al nivel raíz con nombres válidos en formato `img_001_nombre.png`.

6.4.6 Errores de Navegación

- **Crash al pulsar “Editar”:** La app crasheaba con `IllegalArgumentException: No destination with route editar_dieta/123` porque faltaba la ruta en el `NavGraph`. Solución: añadir el composable correspondiente en `NavGraph.kt`.
- **Navegación rota entre pantallas de dieta:** Al añadir un alimento desde `editar_dieta`, se buscaba `getBackStackEntry("crear_dieta")` que no existía en la pila. Solución: pasar los datos por `previousBackStackEntry` en cadena, sin depender de entradas específicas del `backstack`.
- `getBackStackEntry` **sin remember:** Llamar a `getBackStackEntry` durante la composición sin la clave correcta generaba un warning de ciclo de vida. Solución: añadir `remember(navBackStackEntry)` con la entrada como clave.

- **backQueue privado:** Se intentó acceder a `navController.backQueue` para encontrar la pantalla anterior, pero es privado. Solución: usar `navController.previousBackStack`

6.4.7 Errores de Arquitectura y ViewModel

- **Paquete incorrecto tras mover archivos:** Al reorganizar los archivos en subcarpetas, la declaración `package` no se actualizó, causando referencias sin resolver. Solución: sincronizar siempre la primera línea `package` con la ruta de carpetas real.
- **ViewModel sin parámetros para edición:** `CrearDietaViewModel` no aceptaba `dietaId`, por lo que al editar la pantalla aparecía vacía. Solución: añadir `dietaIdEditar: Int?` al constructor y cargar los datos existentes en `init{}` si el id no es nulo.
- **Progreso calculado con ticks en vez de calorías:** Marcar Desayuno/Comida/-Cena rellenaba la barra de progreso aunque no hubiera alimentos registrados. Solución: fijar el progreso a 0 hasta implementar la carga real de alimentos, dejando los ticks como elementos puramente visuales.

6.4.8 Errores de Endpoints

- **Endpoints de DIETA_COMIDA inexistentes:** Faltaban en `ApiService.kt` los endpoints `GET /dieta-comida/dieta/{id}`, `POST /dieta-comida/` y `DELETE /dieta-comida/{id}`. Solución: añadirlos junto con los modelos `DietaComidaRequest` y `DietaComidaResponse`.

Chapter 7

Despliegue

7.1 Arquitectura de Despliegue en AWS

La infraestructura de producción de SmartFit se aloja íntegramente en **Amazon Web Services (AWS)**, utilizando tres servicios diferenciados:

- **Amazon EC2 (t2.micro, Ubuntu 24.04 LTS)**: Instancia virtual donde se ejecuta la API FastAPI con Uvicorn en el puerto 8000. Se le ha asignado una **IP elástica** (100.26.177.220) para garantizar una dirección fija independientemente de los reinicios.
- **Amazon RDS (db.t3.micro, MySQL 8.4)**: Servicio gestionado de base de datos que aloja la BD SmartFit. La EC2 se conecta al RDS mediante la URL de endpoint interna de AWS, sin exposición directa a internet.
- **Amazon S3**: Servicio de almacenamiento de objetos utilizado para alojar las imágenes de ejercicios y alimentos. Se creó un bucket denominado `smartfit-imagenes-dam` en la región `us-east-1`, con acceso público de lectura. Las imágenes se referencian en la base de datos por su nombre de archivo y la aplicación construye la URL completa en tiempo de ejecución:

`https://smartfit-imagenes-dam.s3.us-east-1.amazonaws.com/{nombre}`

Esta arquitectura desacopla el almacenamiento de archivos del servidor de la API, evitando sobrecargar la instancia EC2.

7.2 Requisitos de Instalación

Para desplegar el backend en una instancia EC2 limpia se necesita:

- Ubuntu 22.04 o 24.04 LTS
- Python 3.10 o superior
- pip y venv
- Las siguientes dependencias Python:

```
fastapi
uvicorn
sqlalchemy
pymysql
python-dotenv
passlib[bcrypt]
python-jose[cryptography]
python-multipart
email-validator
bcrypt==4.0.1
```

7.3 Proceso de Despliegue del Backend

1. Conectarse a la instancia EC2 por SSH:

```
ssh -i "smartfit-key.pem" ubuntu@100.26.177.220
```

2. Clonar el repositorio y navegar al directorio de la API:

```
git clone https://github.com/fullace3/Proyecto_Final_DAM.git
cd Proyecto_Final_DAM/BaseDatos/API
```

3. Crear el entorno virtual e instalar dependencias:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Crear el archivo .env con las credenciales de la base de datos.

5. Crear las tablas en la BD:

```
python3 -c "from main import Base, engine;
            Base.metadata.create_all(bind=engine)"
```

6. Arrancar la API en segundo plano:

```
nohup uvicorn main:app --host 0.0.0.0 --port 8000 &
```

7.4 Proceso de Despliegue en Móvil

La aplicación Android se despliega directamente desde Android Studio:

1. Abrir el proyecto en Android Studio.
2. Conectar el dispositivo por USB con depuración USB activada, o iniciar un emulador.
3. Pulsar **Run** → **Run 'app'**.
4. La aplicación se instala automáticamente en el dispositivo.

7.5 Manual Básico de Usuario

1. **Registro:** Al abrir la app por primera vez, pulsar “Regístrate” e introducir nombre, email y contraseña.
2. **Configuración inicial:** Tras el primer login, introducir peso, altura y hora de entrenamiento preferida.
3. **Rutinas:** En la pestaña “Ejercicios”, crear una rutina, añadir ejercicios y empezar a entrenar.
4. **Dieta:** En la pestaña “Dieta”, crear un plan nutricional semanal añadiendo alimentos por día.
5. **Progreso:** En la pestaña “Progreso”, visualizar la evolución del peso y el volumen de entrenamiento.
6. **Notificaciones:** En Perfil → Preferencias → Hora del entrenamiento, configurar el recordatorio diario.

Chapter 8

Planificación y Gestión del Proyecto

8.1 Metodología de Desarrollo

Se ha seguido una metodología iterativa e incremental, desarrollando el proyecto por módulos funcionales. Cada módulo se ha completado de forma vertical (backend + frontend) antes de pasar al siguiente, lo que ha permitido tener versiones funcionales en todo momento.

8.2 Cronograma del Proyecto

Fase	Descripción
Febrero 2026	Diseño de la base de datos, modelo E/R y configuración del entorno de desarrollo.
Marzo 2026	Desarrollo del backend (FastAPI, endpoints REST, autenticación JWT).
Abril 2026	Desarrollo del frontend Android: autenticación, rutinas y ejercicios.
Mayo 2026 (1-2 sem.)	Módulos de dietas, progreso y medidas corporales.
Mayo 2026 (2-3 sem.)	Notificaciones, despliegue en AWS y pruebas.
Mayo 2026 (4 sem.)	Documentación, corrección de errores y preparación de la defensa.

Table 8.1: Cronograma de desarrollo de SmartFit.

8.3 Competencias personales desarrolladas

El desarrollo de SmartFit ha requerido gestionar el tiempo de forma autónoma, priorizando tareas según su impacto y ajustando la planificación cuando surgían

imprevistos técnicos.

8.3.1 Resolución de problemas

Cuando la conexión entre EC2 y RDS dejaba de responder por timeouts silenciosos, en lugar de cambiar de arquitectura, se investigó la causa raíz y se resolvió añadiendo `pool_pre_ping=True` y `pool_recycle=1800` al motor de SQLAlchemy. De forma similar, el crash de navegación con `IllegalArgumentException` se diagnosticó leyendo el stack trace con detenimiento hasta identificar la ruta faltante en el NavGraph.

8.3.2 Comunicación técnica

Durante las tutorías de seguimiento se han explicado decisiones técnicas complejas adaptando el nivel de detalle al interlocutor, como justificar la separación entre `smartfit_config` y `smartfit_session` en función del ciclo de vida de la sesión.

8.3.3 Trabajo autónomo y cumplimiento de plazos

Se ha cumplido el plazo de entrega del 17 de mayo de 2026 siguiendo el cronograma impuesto, compaginando el desarrollo técnico con la redacción de la memoria y la preparación de la defensa.

Chapter 9

Conclusiones

9.1 Objetivos Alcanzados

Se han cumplido los objetivos principales del proyecto:

- Desarrollo de una aplicación Android funcional con Kotlin y Jetpack Compose.
- Implementación de una API REST con FastAPI y despliegue en AWS.
- Base de datos relacional MySQL con más de cuatro tablas interrelacionadas.
- Arquitectura MVVM con separación clara entre capas.
- Notificaciones programadas en hilo secundario mediante WorkManager.
- Soporte de orientación horizontal y modo oscuro/claro.
- Pruebas automatizadas unitarias y de UI.

9.2 Dificultades Encontradas

Las principales dificultades durante el desarrollo han sido:

- La gestión del ciclo de vida de los ViewModels en Jetpack Compose, especialmente al navegar entre pantallas con el mismo tipo de ViewModel.
- La sincronización de estado entre pantallas mediante `SavedStateHandle` en la navegación.
- La configuración del entorno de producción en AWS, especialmente la conectividad entre EC2 y RDS.

9.3 Mejoras Futuras

De cara a futuras versiones, se plantean las siguientes mejoras:

- Implementación de un calendario visual para la planificación de entrenamientos.
- Sincronización en tiempo real del registro diario de comidas con las calorías consumidas.
- Versión web de la aplicación utilizando Kotlin Multiplatform.
- Sistema de roles para entrenadores personales que puedan gestionar las rutinas de sus clientes.
- Integración con wearables para la importación automática de datos de actividad.

Chapter 10

Anexos

10.1 Manual Técnico

10.1.1 Requisitos del Sistema

- **Dispositivo:** Android 8.0 (API 26) o superior.
- **Conexión a internet:** Requerida para comunicarse con la API.
- **Servidor:** Ubuntu 22.04+, Python 3.10+, MySQL 8.0+.

10.1.2 Variables de Entorno

El archivo `.env` del backend requiere las siguientes variables:

```
DATABASE_URL=mysql+pymysql://usuario:contraseña@host:puerto/SmartFit
SECRET_KEY=clave_secreta_jwt
```

10.2 Enlace al Repositorio

El código fuente completo del proyecto está disponible en:

https://github.com/fullace3/Proyecto_Final_DAM

10.3 Referencias y Recursos Utilizados

- Documentación oficial de Jetpack Compose: <https://developer.android.com/jetpack/compose>

- Documentación de FastAPI: <https://fastapi.tiangolo.com>
- Material Design 3: <https://m3.material.io>
- AWS EC2 Documentation: <https://docs.aws.amazon.com/ec2>

Chapter 11

Bibliografía

11.1 Referencias Bibliográficas en Formato APA

- Google LLC. (2024). *Jetpack Compose documentation*. Android Developers. <https://developer.android.com/jetpack/compose> (Consultado: mayo 2026)
- Ramírez, S. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com> (Consultado: marzo 2026)
- JetBrains. (2024). *Kotlin documentation*. <https://kotlinlang.org/docs> (Consultado: febrero 2026)
- Amazon Web Services. (2024). *Amazon EC2 User Guide*. <https://docs.aws.amazon.com/ec2/> (Consultado: mayo 2026)
- Google LLC. (2024). *Material Design 3*. <https://m3.material.io> (Consultado: abril 2026)
- SQLAlchemy. (2024). *SQLAlchemy Documentation*. <https://docs.sqlalchemy.org> (Consultado: marzo 2026)